2.1 L'organisation d'une base de données : le modèle relationnel

Programme

Pédagogie

Compétences attendues

- Interpréter un schéma relationnel
- Vérifier les règles du modèle relationnel
- Adapter un schéma relationnel à un besoin d'évolution d'une base de données

Savoirs associés

- Collecte et sélection de données
- Dépendances fonctionnelles
- Modèle relationnel
- Normalisation des relations et schéma relationnel

Plan

- **Cours** 2.1.1 Qu'est-ce qu'une base de données ? 2.1.2 L'organisation des données 2.1.3 Le modèle relationnel
- **Synthèse**

Mots clés

Attribut • Clé étrangère • Clé primaire • Dépendance fonctionnelle • Dictionnaire des données • Domaine de valeurs • Forme normale • Modèle relationnel • Normalisation • Paramètre • Redondance • Relation • SGBDR • Schéma relationnel • Table de liaison • Tuple

Comment structurer une base de données ? Comment retenir les données pertinentes ? Comment les relier ? L'étude des dépendances fonctionnelles par l'élaboration d'un graphe ou la normalisation des relations permet de construire le modèle relationnel, ossature de la future base de données.

À quoi ça sert ?

Un jour, on vous tendra un fichier de suivi — des ventes, des heures, des immobilisations — et on vous demandera d'en tirer un chiffre. Vous découvrirez qu'il contient trois fois le même client sous trois orthographes, une colonne « Total » que personne ne sait recalculer, et une adresse à jour à un endroit sur deux.

Ce chapitre vous apprend **pourquoi** ce fichier est ainsi, et **comment** on construit une structure qui ne le devient pas. C'est le socle des deux chapitres suivants : on ne peut pas écrire une requête SQL juste sur un modèle faux.

À l'examen, l'exercice classique consiste à lire un schéma relationnel, à repérer ce qui cloche et à le corriger. C'est très exactement ce qui suit.

Cours

2.1.1 Qu'est-ce qu'une base de données ?

Chez Vasseur — la situation

Le suivi des chantiers de Menuiseries Vasseur tient dans un seul tableur. Une ligne par chantier : le client, son adresse, son téléphone, le poseur, le modèle de fenêtre, la quantité, le prix.

Le promoteur Rivière & Fils y apparaît sur 14 lignes. Quand il change d'adresse de facturation, Sofiane Belkacem doit la corriger 14 fois. Le mois dernier, il en a oublié une : une facture est partie à l'ancienne adresse, et elle a été payée avec six semaines de retard.

La question de cette section : qu'est-ce qu'une base de données, et que sait-elle faire qu'un tableur ne sait pas ?

Définition

- Une base de données (ou BDD), c'est un **système organisé** qui permet de :
  - **Stocker** de **grandes quantités** d'informations ;
  - **Organiser** ces informations de manière **structurée** ;
  - Retrouver **facilement** et **rapidement** ce qu'on cherche ;

- **Modifier** ou **supprimer** des données quand c'est nécessaire.

Vous en utilisez tous les jours

- Vous utilisez des bases de données tous les jours sans le savoir :
  - Netflix : stocke vos films, séries, vos préférences ;
  - Facebook/Instagram : vos profils, photos, amis, messages ;
  - Amazon : produits, commandes, clients, stocks ;
  - Le campus : notes, emplois du temps, informations des étudiants.

Deux tables et un lien

- Prenons l'exemple d'une base de données pour un site e-commerce : Table **CLIENTS**

ID	Nom	Prenom	Email
0051	Dupont	Marie	marie.dupont@gmail.com
0127	Martin	Paul	paulmartin@yahoo.fr

Table **COMMANDES**

ID	ID_Client	Date	Montant
0325	0051	15/01/2025	59 €
0326	0127	20/01/2025	120 €

- Les tables sont **reliées entre elles** : ici l'ID_Client fait le lien entre une commande et le client qui l'a passé.

Les principaux acteurs

- Pour travailler avec une base de données, on a besoin de :
  - **Le SGBD** (Système de Gestion de Base de Données) : c'est le logiciel qui gère tout (exemples : MySQL, PostgreSQL, Oracle, SQLite, etc.) ;
  - **Le langage SQL** : pour communiquer avec la base (SELECT, INSERT, UPDATE, DELETE, etc.) ;
  - **L'application** : votre site web ou logiciel qui interroge la base.

> **À retenir**
>
> Une **base de données**, c'est :

- Un entrepôt organisé pour vos données ;
- Rapide pour chercher et récupérer l'info ;
- Sécurisé et fiable ;
- Structuré avec des relations entre les données ;
- Indispensable pour toute application moderne.

Une image pour comprendre

Imaginez une bibliothèque de milliers de livres. Vous pourriez les empiler en vrac dans une pièce : tout y serait, et rien ne serait trouvable. Alors vous les rangez par étagères, par catégories, avec un système de classement et un fichier qui dit où est quoi.

Une base de données, c'est cette bibliothèque. Le tableur, c'est la pile — il fonctionne très bien jusqu'à ce qu'il y ait trop de livres.

Chez Vasseur

Le tableur de suivi des chantiers mélange en réalité **quatre choses différentes** : des clients, des chantiers, des produits et des poseurs. Tant qu'il y a douze lignes, personne ne s'en aperçoit. À 900 lignes, chaque information sur un client est recopiée autant de fois qu'il a de chantiers — et c'est cette recopie, et elle seule, qui a fait partir la facture à la mauvaise adresse.

Une base de données range le client **une fois**, le chantier **une fois**, et se contente de dire lequel va avec lequel.

Chez Vasseur — la réponse

Qu'est-ce qu'une base de données, et que sait-elle faire qu'un tableur ne sait pas ?

C'est un système organisé qui **stocke**, **structure**, permet de **retrouver** et de **modifier** de grandes quantités d'informations — le tout géré par un logiciel spécialisé, le **SGBD**, qu'on interroge en **SQL**.

Ce qu'elle sait faire et que le tableur ne sait pas tient en un mot : **ne pas se répéter**. Dans une base, l'adresse de Rivière & Fils est écrite **une seule fois**, dans une table CLIENTS. Les 14 chantiers ne la contiennent pas : ils contiennent seulement le *numéro* du client. Corriger l'adresse est donc une opération unique, et les 14 chantiers sont à jour instantanément — il n'y a rien à oublier, parce qu'il n'y a rien à répéter.

S'y ajoutent trois choses qu'un tableur partagé tient mal : plusieurs personnes qui écrivent **en même temps** sans s'écraser, des **droits d'accès** par utilisateur, et des **contrôles de cohérence** qui refusent une commande rattachée à un client qui n'existe pas.

Ce n'est donc pas une question de taille de fichier, mais de **structure**. Toute la suite du chapitre consiste à savoir comment on la construit.

✓ **Vérifiez que vous avez compris**

Un collègue affirme : « mon tableur fait la même chose qu'une base de données, il est juste moins joli. » Qu'est-ce qui, précisément, est faux ?

La séparation des données. Un tableur range tout dans **une seule table** : les informations sur le client sont recopiées à chaque ligne où il apparaît. Une base de données répartit l'information en **plusieurs tables reliées** : chaque fait est écrit à un seul endroit.

C'est de là que découle tout le reste — pas de doublon, pas d'incohérence, mise à jour en un seul point. Ajoutez-y le travail simultané à plusieurs, les droits d'accès et les contrôles de cohérence, qu'un tableur ne sait pas assurer.

Mise en pratique

Cas pratique 1 : Le tableur des poseurs

Compétences attendues

- Vérifier les règles du modèle relationnel

En plus du tableur de suivi des chantiers, Nadia Perrot tient un second tableur pour les heur poseurs. Chaque ligne correspond à **une journée d'un poseur sur un chantier**. En voici un ex

Date	Poseur	Tél. poseur	Adresse du chantier	Client	Heures
17/06/2025	Yann Mercier	06 12 34 56 78	5 rue Nationale, Tours	Constructions Ligéria	8
18/06/2025	Yann Mercier	07 55 44 33 22	5 r. Nationale, Tours	Constr. Ligéria	8
02/09/2025	Yann Mercier	06 12 34 56 78	14 rue de la Loire, Blois	Rivière & Fils	7
03/09/2025	Léa Aubry	06 98 76 54 32	14 rue de la Loire, Blois	Rivière & Fils	6,5
17/02/2026	Léa Aubry	06 98 76 54 32	3 allée des Pins, Blois	Belhadj Karim	7,5

Sofiane Belkacem relève trois problèmes :

- Yann Mercier a changé de numéro de portable, et l'ancien subsiste sur une partie des sait plus lequel est le bon ;
- le coût horaire des poseurs est passé de 38 € à 41 € au 1er janvier 2026. La colonne « calculée à la main, et personne ne sait quelles lignes ont été recalculées ;
- quand il filtre sur « 5 rue Nationale, Tours » pour totaliser les heures du chantier de Co Ligéria, il obtient 8 heures au lieu de 16.

Missions

1. **Identifiez les « choses » différentes que ce tableur mélange, et donnez à chacune un**
2. **Expliquez, pour chacun des trois problèmes relevés, en quoi il découle de la recopie de l'information.**
3. **Le coût de la journée doit-il être stocké dans la table des interventions ? Justifiez vot distinguant le coût de la journée et le coût horaire.**
4. **Dans la future base, comment Sofiane obtiendra-t-il les heures du chantier de la rue I sans risque d'en oublier ? Répondez en une ou deux phrases, sans SQL.**

2.1.2 L'organisation des données

☰ Chez Vasseur — la situation

Camille Vasseur a tranché : l'entreprise aura une vraie base. La réunion de lancement tourne court.

Nadia Perrot arrive avec les colonnes de l'atelier, Sofiane Belkacem avec celles de la compta. Tous deux ont une colonne « Réf » — chez l'une c'est la référence du profilé, chez l'autre le numéro de la facture. Élodie Fournier, elle, tient à sa colonne « Total TTC », que plus personne ne sait recalculer.

La question de cette section : quelles données faut-il garder, comment les nommer, et comment savoir lesquelles dépendent des autres ?

Le rôle et le fonctionnement des bases de données

- Une organisation doit pouvoir conserver de façon **structurée** les informations utiles aux différents traitements des processus ;
- Les bases de données relationnelles (BDR), épines dorsales des SI actuels, jouent ce rôle et mémorisent les informations qui y sont décomposées, organisées sous forme de données et stockées ;
- Il est donc possible de **compléter**, de **modifier**, **d'interroger** et **d'exploiter** ces données en fonction des besoins des utilisateurs.

Définition

Le système de gestion de base de données relationnelle (SGBDR) prend en charge différentes bases de données qui interagissent.

La collecte et la sélection des données

Les objectifs de la collecte

- Il est inutile de stocker, dans une base de données, des informations qui ne seront pas mises en œuvre dans un processus ou qui n'interviendront pas dans un traitement ;
- Il faut donc choisir les données utiles en réalisant des entretiens, en analysant des documents ;
- Le travail d'analyse peut mettre en évidence des ambiguïtés à traiter par :
  - L'individualisation des polysèmes (données qui ont le même nom mais pas la même signification) en attribuant deux libellés différents ;
  - La conservation d'une donnée pour les synonymes (données qui ont la même signification mais des noms différents dans plusieurs documents).

À savoir

Le recensement des règles de gestion pour l'obtention des données calculées est important : il permet de vous assurer que vous n'oubliez pas de données élémentaires. Vous pouvez éventuellement choisir de conserver une donnée calculée pour répondre à un impératif de gestion.

Exemple

Le code du client et celui du fournisseur ne peuvent être tous deux référencés par le nom « Code ». La référence et le code d'un produit représentent la même donnée que l'on peut nommer « référence_produit ». Les données doivent être élémentaires : elles doivent être utiles en l'état, sans être scindées et ne pas résulter d'un calcul à intégrer à un traitement.

Exemple

Une adresse est le plus souvent séparée en trois éléments, l'adresse_rue (10 rue Amiral Courbet), l'adresse_ville (LORIENT) et le code_postal (56100). Le montant TTC d'un

produit peut être retrouvé en appliquant un taux de TVA au montant HT. Ce taux ne sera pas conservé.

Le recueil des informations et l'élaboration du dictionnaire des données

Définitions

- Un **dictionnaire des données** est un document qui regroupe les données élémentaires et constitue donc un référentiel pour élaborer la base, il la décrit ;
- Un **paramètre** est une donnée susceptible d'intervenir dans les calculs, dont la valeur est stable (à moyen terme) et unique.

- Les paramètres peuvent être clairement identifiés dans le dictionnaire alors que les constantes (données permanentes) ne le sont pas.

Exemple

Le dictionnaire reprend le taux de TVA (paramètre) mais pas l'adresse de l'entreprise ou sa désignation (constantes).

- **Forme**
  - Le dictionnaire des données prend le plus souvent la forme d'un tableau à quatre colonnes contenant le nom, le code et le type de donnée ainsi que d'éventuels commentaires ;
  - Le type décrit la nature des valeurs associées à une donnée ;
  - Plusieurs typologies existent, il conviendra a minima de différencier les types numériques (réels, entiers, monétaires, dates…), alphabétiques (ou alphanumériques) et booléens (logiques).
- **Établissement du dictionnaire**
  - Il est créé en respectant trois étapes :
    - Le recensement des données. Il consiste à attribuer à chaque donnée un code et un nom significatif ;
    - L'indication du type de chaque donnée et de la règle de calcul éventuelle ou de toute autre observation utile ;
    - La sélection des données élémentaires par la résolution des ambiguïtés éventuelles et la suppression des données calculées et des paramètres.

Exemple

La SARL Magadur a adressé à sa cliente la facture suivante.

Facture MAR257 du 17/03/N

SARL Magadur — 31 rue des Goélands, 56340 Plouharnel

Cliente n° 411256 : Zoé Huet, 10 rue Amiral Courbet, 56100 Lorient — tél. 02 97 27 56 43 — zoe.huet@bidule.bzh

Référence	Désignation	Quantité	PU HT	Montant
MG02	Mélange de graines et de fruits secs	50	15,00	750,00
LA03	Lait d'avoine	5	1,50	7,50

Total HT : 757,50 € — **TVA (5,5 %)** : 41,66 € — **Total TTC** : 799,16 €

Le dictionnaire des données correspondant à la facture de l'entreprise Magadur est le suivant :

Code donnée	Nom Donnée	Type	Commentaire
NumCli	Numéro du client	Numérique	Séquentiel
NomCli	Nom du client	Texte	
PrenomCli	Prénom du client	Texte	
AdrRueCli	Adresse rue du client	Texte	
CPCli	Code postal du client	Texte	5 caractères
AdrVilleCli	Adresse ville du client	Texte	
TelCli	Téléphone du client	Texte	Format 33-/—/—/—/—
MailCli	mail du client	Texte	
NumFact	Numéro de la facture	Texte	

Code donnée	Nom Donnée	Type	Commentaire
DateFact	Date facture	Date	Format date jj/mm/aa
RefProd	Référence du produit	Texte	
Désignation	Désignation du produit	Texte	
Quantité	Quantité du produit	Numérique	Réel, deux décimales
PUHT	Prix unitaire HT du produit	Numérique	Réel, deux décimales
TauxTVA	Taux de TVA	Numérique	Réel, deux déci

Seules les données utilisables dans les calculs et les numéros séquentiels (relatifs à des éléments qui se suivent dans un ordre préétabli) sont de type numérique. Le taux de TVA est un paramètre, il n'est pas inclus dans le dictionnaire mais aurait pu l'être. Les montants par produits, la TVA et les totaux sont calculés.

Conformément à la règle, ils ne sont pas identifiés dans ce dictionnaire car ce ne sont pas des données à stocker.

La structuration des données

Les objectifs de la structuration

- Les données élémentaires doivent être organisées pour être traitées aisément ;
- Cette organisation doit être pertinente et mettre en évidence les liens entre les données pour obtenir les informations nécessaires à l'organisation ;
- Elles forment ensemble, quand elles se complètent, des objets matériels ou immatériels du système d'information ;
- L'utilisation d'une structure de données appropriée simplifie les bases qui en découlent, ce qui contribue à supprimer les erreurs et anomalies ;
- La future base, ainsi structurée, pourra alors traiter les données automatiquement de façon rapide et efficace.

Les attributs

> **Définition**
>
> Un attribut correspond au plus petit élément d'information manipulable.

- Un nom d'attribut permet de décrire les données élémentaires, elles prennent alors des valeurs et un sens dans le système d'information (SI) étudié ;
- L'organisation des attributs permet donc de structurer les données.

> **Exemple**
>
> L'attribut `NomCli` prend diverses valeurs comme ici « Huet », il caractérise le client et a donc une signification dans le SI de l'entreprise Magadur.

- L'ensemble des valeurs possibles pour un attribut représente le domaine de valeurs.

Les dépendances fonctionnelles

- **Pourquoi parler de dépendances fonctionnelles ?**
  - Imaginez une base de données comme un tableau Excel géant où vous stockez des informations sur vos clients, vos produits ou vos commandes. Pour que cette base soit fiable et facile à utiliser, il faut organiser les données de manière logique ;
  - Les dépendances fonctionnelles (DF) sont des règles qui permettent de :
    - Éviter les redondances (ex. : répéter 10 fois l'adresse d'un même client) ;
    - Garantir la cohérence (ex. : si le code postal change, il doit être mis à jour partout) ;
    - Optimiser la structure de la base pour des requêtes plus rapides.

> **Définition**
>
> Une dépendance fonctionnelle est une relation entre deux attributs (colonnes) d'une table. On note : **X** → **Y** (se lit "X détermine Y" ou "Y dépend fonctionnellement de X").

> **Exemple**
>
> Prenons une table `Clients` avec les colonnes :
>
> - `N°Client` (ex. : C001, C002)
> - `Nom` (ex. : Dupont, Martin)
> - `Ville` (ex. : Paris, Lyon) Ici, `N°Client → Nom` :
> - Si je connais le `N°Client` (ex. : C001), je peux trouver un seul `Nom` associé (ex. : Dupont).
> - À l'inverse, `Nom → N°Client` n'est pas valide : deux clients peuvent s'appeler "Dupont" !

- **Règles clés à retenir**
  - Pour qu'une dépendance **X** → **Y** soit valide :
    1. **Unicité** : à chaque valeur de **X** correspond **une seule valeur de Y** (pas d'ambiguïté).
       - *Exemple valide* : `N°Client → Ville` (un client a une seule ville) ;
       - *Exemple invalide* : `Ville → N°Client` (une ville peut avoir plusieurs clients).
    2. **Stabilité dans le temps** : la relation doit toujours être vraie, même si les données changent.
       - *Exemple *: `CodePostal → Ville` est généralement stable (un code postal = une ville).
    3. **Minimalité** : **X** doit être le plus petit ensemble possible pour déterminer **Y**.
       - *Exemple *: si `N°Client + Nom → Ville` mais que `N°Client → Ville` suffit, la première DF n'est pas minimale.
- **A quoi ça sert ?**
  - Les dépendances fonctionnelles sont utilisées pour normaliser une base de données, c'est-à-dire l'organiser en tables sans anomalies ;

Exemple

N° Commande	N°Client	NomClient	Ville Client	Produit
CMD1	C001	Dupont	Paris	Stylo
CMD2	C001	Dupont	Paris	Cahier
CMD3	C002	Martin	Lyon	Gomme

Anomalies :

- **Redondance** : "Dupont" et "Paris" sont répétés pour chaque commande du client C001 ;
- **Risque d'incohérence** : si Dupont déménage à Marseille, il faut modifier toutes ses lignes. **Solution : appliquer les DF** :

1. On identifie les DF :

- `N°Client → NomClient, VilleClient` (un client a un nom et une ville uniques) ;

- `N°Commande → N°Client`, Produit (une commande est liée à un client et un produit).

2. On sépare en **2 tables** :

- Table `Clients` :

N°Client	NomClient	VilleClient
C001	Dupont	Paris
C002	Martin	Lyon

- Table `Commandes` :

N°Commande	N°Client	Produit
CMD1	C001	Stylo
CMD2	C001	Cahier
CMD3	C002	Gomme

Avantages :

- Plus de redondance ;
- Mise à jour facile (ex. : changer la ville de Dupont en une seule fois).

- **Résumé**

Concept	Définition	Exemple
Dépendance X → Y	Y est déterminé de manière unique par X	`N°Client → Ville`
Normalisation	Organisation des données pour éviter redondances et anomalies	Séparer Clients et Commandes.

Concept	Définition	Exemple
Clé primaire	Attribut (ou groupe) qui identifie uniquement une ligne	`N°Client`

Le graphe des dépendances fonctionnelles

- Une fois les dépendances repérées une à une, il faut les voir **toutes ensemble**. C'est le rôle du **graphe des dépendances fonctionnelles** (graphe des DF) : un simple dessin où chaque attribut est un point, et chaque dépendance une flèche.

> **Définition**
>
> Le **graphe des dépendances fonctionnelles** est la représentation, sous forme de flèches, de l'ensemble des dépendances fonctionnelles relevées entre les attributs du dictionnaire des données. C'est l'étape intermédiaire entre le dictionnaire et le modèle relationnel.

> **Une image pour comprendre**
>
> C'est un plan de métro. Chaque station est un attribut ; chaque ligne indique qu'en partant d'ici, on arrive nécessairement là. Les stations d'où **partent** des lignes sans qu'aucune n'y arrive sont les têtes de réseau : ce sont elles qui deviendront les clés primaires. Les stations qui ne font que **recevoir** sont de simples attributs.

- **Comment le construire**, en quatre gestes :
  1. **Poser les attributs** du dictionnaire des données, chacun une fois, en les espaçant ;
  2. **Tracer une flèche** de X vers Y chaque fois que connaître X suffit à connaître une seule valeur de Y. Se poser la question dans les deux sens : la flèche inverse est presque toujours fausse ;
  3. **Repérer les sources** — les attributs d'où partent des flèches sans qu'aucune n'aboutisse sur eux. Ce sont les futures **clés primaires** ;
  4. **Traiter les cas à deux sources** : quand un attribut n'est déterminé que par un **couple** d'attributs, la source est ce couple. On l'entoure : ce sera une **clé primaire composée**.

> **Exemple**
>
> Sur le dictionnaire de l'entreprise Magadur vu ci-dessus, le graphe se lit ainsi :

```
NumCli  ⟶ NomCli, PrenomCli, AdrRueCli, CPCli, AdrVilleCli, TelCli,
MailCli
NumFact ⟶ DateFact
NumFact ⟶ NumCli
RefProd ⟶ Désignation, PUHT
(NumFact, RefProd) ⟶ Quantité
```

Trois attributs sont **sources** et ne sont la cible d'aucune flèche depuis un attribut simple : `NumCli`, `NumFact` et `RefProd`. Ils deviendront trois clés primaires.

La flèche `NumFact ⟶ NumCli` relie deux sources entre elles : elle annonce une **clé étrangère**.

Et `Quantité` n'est déterminée ni par la facture seule (il y a plusieurs produits dessus), ni par le produit seul (il figure sur plusieurs factures) : sa source est le **couple**. D'où une quatrième relation, à clé primaire composée.

⚠ L'erreur classique

Tracer une flèche parce que, dans les données du jour, la correspondance se vérifie. `NomCli ⟶ NumCli` est vraie tant qu'il n'y a qu'un seul Dupont. Une dépendance fonctionnelle doit être vraie **par nature et pour toujours**, pas seulement dans le jeu d'exemples qu'on a sous les yeux. Demandez-vous toujours : « pourrait-il un jour y en avoir deux ? »

☰ Chez Vasseur — la réponse

Quelles données faut-il garder, comment les nommer, et comment savoir lesquelles dépendent des autres ?

Ce qu'on garde : les données **élémentaires** et utiles à un traitement. On écarte donc la colonne « Total TTC » d'Élodie — c'est une **donnée calculée**, qui se retrouve à partir de la quantité, du prix unitaire et du taux de TVA. La stocker, c'est créer un chiffre qui pourra un jour contredire les autres. Le taux de TVA, lui, est un **paramètre** : stable, unique, il ne va pas dans le dictionnaire.

Comment on les nomme : en levant les ambiguïtés. Les deux colonnes « Réf » de Nadia et Sofiane sont un **polysème** — même nom, sens différents : elles deviennent `RefProfile` et

`NumFacture`. À l'inverse, si l'atelier dit « référence » là où la compta dit « code article » pour la même chose, c'est un **synonyme** : on n'en garde qu'un. Tout cela se consigne dans le **dictionnaire des données** : code, nom, type, commentaire.

Comment on sait ce qui dépend de quoi : par les **dépendances fonctionnelles**, rassemblées dans un **graphe**. Connaître le numéro de chantier suffit-il à connaître son adresse ? Oui — flèche. Connaître l'adresse suffit-il à connaître le chantier ? Non, il peut y en avoir eu deux au même endroit — pas de flèche.

La réunion aurait dû commencer par là : non pas « quelles colonnes veux-tu ? », mais **« de quoi part-on pour connaître le reste ? »**.

✓ **Vérifiez que vous avez compris**

Faut-il faire figurer l'attribut « âge du salarié » dans le dictionnaire des données ?

Non. L'âge est une **donnée calculée** : il se déduit de la date de naissance et de la date du jour. Le stocker obligerait à le recalculer chaque année pour chaque salarié — et à vivre avec des âges faux entre deux recalculs.

On conserve la **date de naissance**, qui est élémentaire et stable, et l'âge se calcule au moment où on en a besoin. C'est la règle générale : *on ne stocke jamais ce qu'on sait recalculer.*

Mise en pratique

Cas pratique 2 : Le devis d'Élodie

Compétences attendues

- Interpréter un schéma relationnel
- Vérifier les règles du modèle relationnel

Avant de construire la base, l'équipe recense les données des documents existants. Voici un devis établi par Élodie Fournier à l'agence de Blois :

```
MENUISERIES VASSEUR — Agence de Blois            Devis n° D-2026-0412
Date : 14/04/2026                                Validité : 30 jours
Établi par : Élodie Fournier
Client : M. Karim Belhadj — 3 allée des Pins, 41000 Blois — 06 21 43 65 87
Réf. : CL-0007
Adresse du chantier : 3 allée des Pins, 41000 Blois

 Réf.    Désignation          Matière   Qté    P.U. HT    Total HT
 F-510   Fenêtre 2 vantaux    PVC         4     265,00    1 060,00
 F-520   Fenêtre 1 vantail    PVC         3     198,00      594,00
 P-530   Porte d'entrée       PVC         1     540,00      540,00
                                                Total HT   2 194,00
                                                TVA 10 %     219,40
                                                Total TTC  2 413,40
```

Missions

1. **Établissez le dictionnaire des données de ce devis** (code, désignation, type, commentaire), en écartant les données calculées et les paramètres. Justifiez chaque exclusion.
2. **Une des mentions du devis pose un problème de polysémie. Repérez-la et proposez une correction.**
3. **Relevez les dépendances fonctionnelles et représentez le graphe des DF.**
4. **Un attribut n'est déterminé que par un couple d'attributs. Lequel, et pourquoi ?**
5. **Élodie accorde parfois une remise sur un produit, pour un devis donné. Qu'est-ce que cela change à la dépendance du prix unitaire ?**

2.1.3 Le modèle relationnel

☰ Chez Vasseur — la situation

Le dictionnaire est prêt, le graphe aussi. Reste à décider ce qu'on construit vraiment : combien de tables, contenant quoi, et reliées comment.

Une question bloque l'équipe. Un chantier comporte plusieurs modèles de fenêtres, et un même modèle équipe plusieurs chantiers. Où écrit-on alors « 14 fenêtres du modèle F-210 pour le chantier de Saint-Avertin » ? Dans la table des chantiers, il faudrait 14 colonnes. Dans celle des produits, autant que de chantiers.

La question de cette section : comment passe-t-on d'une liste de données à un jeu de tables reliées, et à quoi reconnaît-on qu'un découpage est bon ?

D'abord, une table

- Le modèle relationnel, c'est **une méthode d'organisation des données** inventée en 1970 par **Edgar Frank Codd**, informaticien chez IBM ;
- L'idée principale ? **Organiser les données dans des tables, et créer des relations entre ces tables**.

1. **Les tables** (ou relations)

- Une table, c'est comme un tableau avec :
  - Des **lignes** (appelées tuples ou enregistrements) : 1 ligne = 1 entité ;
  - Des **colonnes** (appelées attributs ou champs) : les caractéristiques.

Exemple : table ETUDIANTS

ID_Etudiant	Nom	Prenom	Date_Naissance	Classe
1	Dubois	Julie	12/03/2004	ENGDE 1
2	Mercier	Thomas	25/07/2003	ENGDE 1
3	Bernard	Lisa	18/11/2004	ENGDE 2

2. **Les clés**

- La clé primaire :
  - C'est **l'identifiant unique** de chaque ligne. Pas de doublon possible !
  - Dans notre table ETUDIANTS, c'est **ID_Etudiant**. Deux étudiants peuvent avoir le même nom, mais jamais le même ID.
- La clé étrangère :
  - C'est une colonne qui **fait référence à la clé primaire d'une autre table**. C'est elle qui crée les **relations** entre les tables.

3. **Les relations**

- C'est le cœur du modèle ! Les tables ne sont pas isolées, elles communiquent entre elles.
  Exemple : un système scolaire ❌ MAUVAISE approche (tout dans une table)

ID	Nom	Prenom	Matière	Note	Coef
1	Dubois	Julie	Mathématiques	15	3
2	Dubois	Julie	Informatique	17	4
3	Dubois	Julie	Anglais	12	2
4	Mercier	Thomas	Mathématiques	10	3
5	Mercier	Thomas	Informatique	4	4

- **Problèmes** :
  - Répétition des noms (redondance) ;
  - Si Julie change de nom → modifier plein de lignes ;
  - Perte d'espace disque ;
  - Risque d'erreurs (incohérence). ✅ BONNE approche (modèle relationnel)
    Table **ETUDIANTS**

ID_Etudiant	Nom	Prenom	Classe
1	Dubois	Julie	ENGDE 1
2	Mercier	Thomas	ENGDE 1

Table **MATIERES**

ID_Matiere	Nom_Matiere	Coefficient
1	Mathématiques	3
2	Informatique	4
3	Anglais	2

Table **NOTES** (table de liaison)

ID_Note	ID_Etudiant	ID_Matiere	Note	Date
1	1	1	15	10/01/2025
2	1	2	17	12/01/2025
3	1	3	12	15/01/2025

ID_Note	ID_Etudiant	ID_Matiere	Note	Date
4	2	1	10	10/01/2025
5	2	2	14	12/01/2025

- **Avantages :**
  - Pas de répétition ;
  - Modification facile (un seul endroit) ;
  - Gain d'espace ;
  - Cohérence garantie.

Les grands principes

Deux vocabulaires pour les mêmes objets

Vous rencontrerez les deux, souvent dans la même copie d'examen. Ils désignent exactement la même chose : le premier est celui de la théorie, le second celui des logiciels.

Théorie (modèle relationnel)	Pratique (SGBD, tableur)
Relation	Table
Tuple, n-uplet	Ligne, enregistrement
Attribut	Colonne, champ
Domaine de valeurs	Type de la colonne
Degré	Nombre de colonnes

Les pages qui suivent emploient le vocabulaire de la théorie, parce que c'est celui du programme et celui de l'énoncé d'examen.

Définitions

- Un **modèle** est une simplification de la réalité qui facilite sa compréhension. Le modèle doit être clair, rigoureux et aisément interprétable ; il intègre les éléments fondamentaux qui expliquent un phénomène ou une situation ;

- Développé par l'informaticien britannique Edgar Frank Codd (1923-2003), le **modèle relationnel** représente l'ensemble des relations d'un système d'information.

- Le modèle relationnel peut être élaboré à partir du graphe des dépendances fonctionnelles.

Les relations

- Un ensemble d'attributs ayant la même source de dépendance fonctionnelle, et donc associés de façon significative, constitue une relation ;
- Élément du système d'information, une relation peut être représentée par un tableau à deux dimensions ou par un **schéma relationnel** ;
- L'attribut source de dépendance fonctionnelle est appelé « **clé primaire** » ; il est positionné en première position et souligné dans la présentation schématique ;
- **Représentation sous forme de tableau** :
  - Le tableau comporte un nombre non défini de colonnes qui correspondent, chacune, à un attribut, chacun étant associé à un nom unique ;
  - Chaque ligne du tableau est un tuple ou n-uplet, c'est-à-dire une occurrence de valeurs d'attribut ;
  - Une valeur de clé primaire permet d'identifier, de façon certaine, le tuple correspondant, sachant qu'elle est unique.

Exemple Relation PRODUIT

RefProd	Désignation	PUHT
AV01	Avoine	6,50
…		
LA03	Lait d'avoine	1,50
…		
MG02	Mélange de graines et de fruits secs	15,00
…		

- **Représentation sous forme de schéma** :
  - Le schéma est composé du nom de la relation, suivi de la liste des **attributs** qui la composent.

Exemple

Le schéma de la relation précédente devient, en notation fonctionnelle : `PRODUIT (RefProd, Désignation, PUHT)`

- Si la clé primaire d'une relation est en dépendance fonctionnelle avec la clé primaire d'une autre relation, la clé primaire « BUT » devient une clé étrangère dans la relation dont la clé primaire est la « SOURCE » de la dépendance fonctionnelle ;
- Chaque clé étrangère est le plus souvent signalée par un dièse (#) ;
- Elle est reprise dans la relation SOURCE, généralement à la suite des autres attributs.

Comment lire les modèles de ce cours

Dans les manuels, la clé primaire est **soulignée**. Le soulignement passe mal à l'écran et n'est pas restitué par les lecteurs d'écran : ce cours emploie donc une notation explicite.

- la **clé primaire** est le ou les attributs de tête ; quand elle est composée, elle est **rappelée en toutes lettres sous le modèle** ;
- un attribut précédé d'un `#` est une **clé étrangère** ;
- un attribut peut être **à la fois** clé étrangère et membre de la clé primaire.

Sur votre copie d'examen, soulignez. C'est la notation attendue par le correcteur.

Exemple

La reprise de la clé étrangère dans la relation SOURCE donne :

```
FACTURE (NumFact, DateFact, #NumCli)
```

`NumFact` est la **clé primaire** de FACTURE. `#NumCli` est la **clé étrangère** : c'est la clé primaire de CLIENT, recopiée dans FACTURE pour relier les deux relations.

Les caractéristiques du modèle relationnel

- Le modèle relationnel regroupe toutes les relations du système d'information étudié ;
- Il possède diverses caractéristiques et contraintes :

    - Le nom de chaque relation est unique ;
    - Le degré de la relation représente son nombre d'attributs ;
- Une relation répond à différentes contraintes :
    - Elle possède une clé dite primaire qui permet d'identifier les différents tuples, celle-ci peut correspondre à un seul attribut ou être composée de plusieurs attributs (concaténés) ;
    - Chaque tuple est unique ;
    - Aucune clé primaire ne peut être nulle.
- Toutes les relations doivent être liées, c'est le rôle des clés étrangères.

Exemple

Le modèle relationnel de gestion des factures client, construit à partir du graphe des dépendances fonctionnelles, donne :

```
PRODUIT  (RefProd, Désignation, PUHT)
FACTURE  (NumFact, DateFact, #NumCli)
CLIENT   (NumCli, NomCli, PrenomCli, AdrRueCli, AdrVilleCli, CPCli, TelCli,
MailCli)
FACTURER (#NumFact, #RefProd, Quantité)
```

La relation `FACTURER` possède une **clé primaire composée**, qui découle des dépendances fonctionnelles : l'attribut « Quantité » ne peut être rattaché à une source unique — une quantité n'a de sens que pour un produit *donné* sur une facture *donnée*.

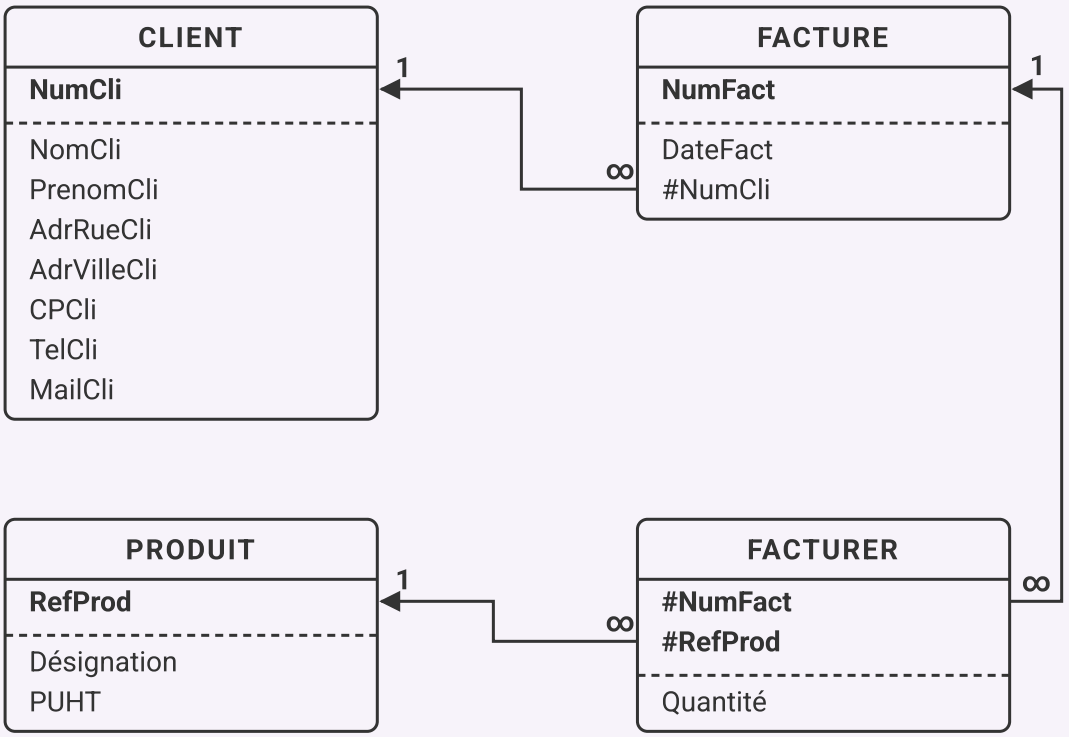
C'est cette relation qui relie `FACTURE` et `PRODUIT`, puisque sa clé primaire est formée du regroupement (concaténation) des clés primaires de ces deux relations.

- **Représentation sous forme de graphique** :
    - Les SGBDR permettent d'implanter le modèle relationnel dans une base de données sous forme graphique.

Exemple

Représenté sous forme graphique, le **modèle relationnel** de gestion des factures client donne :

Chaque flèche va d'une clé étrangère (#) à la clé primaire qu'elle désigne



En gras, au-dessus du pointillé : la clé primaire.
1 … ∞ : un client a plusieurs factures, une facture n'a qu'un client.

Description du schéma, en toutes lettres

Quatre tables. `FACTURE` porte la clé étrangère `#NumCli`, qui désigne la clé primaire `NumCli` de `CLIENT` : un client a plusieurs factures (∞), une facture n'a qu'un client (1).

`FACTURER` a pour clé primaire le couple (`#NumFact`, `#RefProd`), dont chaque membre est une clé étrangère : `#NumFact` désigne `FACTURE`, `#RefProd` désigne `PRODUIT`. Une facture porte plusieurs produits, un produit figure sur plusieurs factures.

Un logiciel de bases de données comme Access dessine le même schéma dans sa fenêtre « Relations » : une petite clé devant chaque clé primaire, et sur chaque lien un **1** du côté de la clé primaire et un **∞** du côté de la clé étrangère.

Une image pour comprendre

La clé étrangère, c'est le numéro de dossier qu'on porte sur un document. La facture ne recopie pas l'identité complète du client : elle porte son *numéro*, et c'est ce numéro qui

permet d'aller chercher le reste. Si le client déménage, on change son adresse dans un seul dossier — toutes les factures qui portent son numéro pointent désormais vers la bonne.

C'est précisément ce qui manquait au tableur de suivi de chantiers.

Les types de relations

- Relation 1:n (Un vers Plusieurs)
  - **Un** étudiant peut avoir **plusieurs** notes, mais **une** note appartient à un **seul** étudiant ;
  - `ETUDIANTS (1) ----< (N) NOTES`
- Relation n:n (Plusieurs vers Plusieurs)
  - **Plusieurs** étudiants peuvent suivre **plusieurs** matières ;
  - `ETUDIANTS (N) ----< INSCRIPTIONS >---- (M) MATIERES`
  - On crée alors une **table intermédiaire** (table de liaison).
- Relation 1:1 (Un vers Un)
  - **Un** étudiant a **un seul** dossier médical, et **un** dossier appartient à **un seul** étudiant ;
  - `ETUDIANTS (1) ---- (1) DOSSIERS_MEDICAUX`

⁝☰ Chez Vasseur

Les trois cas se lisent directement dans l'entreprise :

- **1:1** — un salarié a un seul contrat de travail en cours, et un contrat concerne un seul salarié ;
- **1:n** — un client passe plusieurs commandes, mais une commande donnée appartient à un seul client. La clé étrangère `#NumClient` se met dans COMMANDE, du côté « plusieurs » ;
- **n:n** — un chantier comporte plusieurs modèles de fenêtres, et un modèle équipe plusieurs chantiers. Impossible de loger le lien d'un côté ou de l'autre : il faut une **table de liaison**.

C'est la réponse à la question qui bloquait l'équipe. On crée `POSER (#NumChantier, #RefProduit, Quantité)`. Sa clé primaire est le **couple** — un chantier *et* un produit — et la quantité y trouve enfin sa place, parce qu'elle ne dépend ni de l'un ni de l'autre pris isolément.

⚠ Le réflexe à acquérir

Toute association n:n donne naissance à une table. C'est la règle la plus rentable de ce chapitre : dès qu'un énoncé dit « plusieurs … peuvent … plusieurs », vous savez qu'il manque une table, et vous savez que sa clé primaire sera composée des deux clés étrangères.

Du graphe au modèle relationnel

- Trois règles permettent de passer du graphe des DF au modèle relationnel :
  - L'existence d'un attribut source de dépendances fonctionnelles indique une relation dont cet attribut est la clé primaire ;
  - Une dépendance fonctionnelle composée indique une relation dont la clé primaire est concaténée. Elle sera formée de tous les attributs combinés dans la source de DF. Elle contiendra éventuellement d'autres attributs ;
  - Une dépendance fonctionnelle entre deux attributs source de DF révèle une clé étrangère (attribut but) dans la relation dont l'attribut source est la clé primaire.

Les formes normales

- Les formes normales sont des règles pour organiser les tables et éviter les anomalies ;
- Elles s'appuient sur les dépendances fonctionnelles.
- **Première forme normale (1FN)** :
  - **Règle** :
    - Chaque **colonne** doit contenir **une seule valeur** (pas de listes ou de valeurs multiples) ;
    - Chaque **ligne** doit être **unique** (identifiée par une clé primaire).

☰ Exemple

Non-conforme :

IDClient	NomsEnfants
C001	"Lucas, Emma"

Conforme (1FN) :

IDClient	NomsEnfants
C001	Lucas
C001	Emma

- **Deuxième forme normale (2FN)** :
  - **Conditions** :
    - La table est en **1FN** ;
    - **Tous les attributs non-clés dépendent de la totalité de la clé primaire** (pas de dépendance partielle) ;
    - Cela signifie essentiellement qu'il doit y avoir une relation directe entre chaque colonne et la clé primaire, et non entre les autres colonnes.

☰ Exemple

Tables avec **clé primaire composée** (plusieurs colonnes). ***Non-conforme*** : Table `Commandes` (clé primaire : `IDCommande + IDProduit`) :

IDCommande	IDProduit	Quantite	NomClient	AdresseClient
CMD1	P1	2	Dupont	Paris
CMD1	P2	1	Dupont	Paris

Problème :

- `NomClient` et `AdresseClient` dépendent uniquement de `IDCommande` (pas de `IDProduit`) ;
- **Dépendance partielle** : `IDCommande → NomClient` (mais la clé primaire est `IDCommande + IDProduit`). ***Conforme (2FN)*** :

1. Table `Commandes` (clé : `IDCommande + IDProduit`) :

IDCommande	IDProduit	Quantite	IDClient
CMD1	P1	2	C001

IDCommande	IDProduit	Quantite	IDClient
CMD2	P2	1	C001

2. Table `Clients` (clé : `IDCommande`) :

IDClient	NomClient	AdresseClient
C001	Dupont	1 Rue de Paris

- **Troisième forme normale (3FN)** :
  - **Conditions** :
    - La table est en **2FN** ;
    - **Aucune dépendance fonctionnelle transitive** entre attributs non-clés.

⁝☰ **Exemple**

Non-conforme : Table `Etudiants` (clé primaire : `IDEtudiant`) :

IDEtudiant	Nom	IDDepartement	NomDepartement
E1	Martin	D10	Comptabilité
E2	Durand	D10	Comptabilité

Problème :

- `NomDépartement` dépend de `IDDépartement`, pas directement de `IDEtudiant` ;
- **Redondance** : “Comptabilité” est répété pour tous les étudiants de D10. ***Conforme (3FN)*** :

1. Table `Etudiants` :

IDEtudiant	Nom	IDDepartement
E1	Martin	D10

IDEtudiant	Nom	IDDepartement
E2	Durand	D10

2. Table `Departements` :

IDDepartement	NomDepartement
D10	Comptabilité

- **Résumé des formes normales**

Forme normale	Règle principale	Exemple de correction
1FN	Pas de valeurs multiples par cellule	Séparer “Lucas, Emma” en 2 lignes
2FN	Pas de dépendance partielle à la clé	Séparer `NomClient` de `N°Commande+Produit`
3FN	Pas de dépendance transitive	Séparer `NomDépartement` de `N°Étudiant`

L’intégrité référentielle

Une clé étrangère n’est pas une simple colonne : elle engage la base. Si un chantier porte le numéro de client 9 alors qu’aucun client 9 n’existe, ce chantier n’appartient à personne, et toutes les requêtes qui rapprochent chantiers et clients le feront disparaître.

> **Définition**
>
> L’**intégrité référentielle** est la règle selon laquelle **toute valeur d’une clé étrangère doit exister comme clé primaire** dans la table à laquelle elle renvoie. Le SGBD la fait respecter de lui-même.

Sur la base de Menuiseries Vasseur, que vous interrogerez au **chapitre 2.3**, le chantier porte le numéro de son client. Le SGBD **refuse** donc :

- d'**enregistrer un chantier pour un client qui n'existe pas** ;
- de **supprimer un client qui a encore des chantiers** : Constructions Ligéria porte les chantiers 105 et 106, qui deviendraient orphelins.

Il **accepte** en revanche de supprimer SCI Le Clos, qui n'a aucun chantier.

> **À savoir**
>
> Ces refus ne sont pas des pannes : ils protègent la cohérence des données. Vous les verrez se produire pour de bon dans la console SQL du chapitre 2.3, avec les requêtes de mise à jour.

Les règles d'une base bien conçue

- Pour qu'une base soit bien conçue, elle doit respecter certaines règles :
  - Chaque table a une clé primaire unique ;
  - Pas de valeurs multiples dans une cellule :
    - "Math, Info, Anglais" dans une seule case ;
    - Une ligne par matière.
- Pas de répétition de groupes de colonnes :
  - Note1, Note2, Note3… ;
  - Une table NOTES séparée.
- Les données dépendent de la clé primaire (normalisation)

Modèle récapitulatif

```
ETUDIANTS (ID_Etudiant, Nom, Prenom, Classe)
MATIERES  (ID_Matiere, Nom_Matiere, Coefficient)
NOTES     (ID_Note, #ID_Etudiant, #ID_Matiere, Note, Date)
```

L'intérêt du modèle relationnel

- Le modèle relationnel permet de :
  - Éviter les redondances (pas de répétition) ;
  - Garantir l'intégrité (les données restent cohérentes) ;
  - Rechercher efficacement (grâce aux liens entre tables) ;
  - Modifier facilement (un changement à un seul endroit) ;
  - Croiser les informations (requêtes SQL complexes).

À retenir

Le modèle relationnel, c'est :

- Des tables structurées ;
- Des clés pour identifier et relier ;
- Des relations entre tables ;
- Une organisation logique et efficace.

Chez Vasseur — la réponse

Comment passe-t-on d'une liste de données à un jeu de tables reliées, et à quoi reconnaît-on qu'un découpage est bon ?

Le passage se fait par le graphe des dépendances fonctionnelles, selon trois règles :

- chaque **attribut source** de dépendances donne une relation, dont il est la clé primaire ;
- une **source composée** donne une relation à clé primaire concaténée ;
- une dépendance **entre deux sources** donne une clé étrangère.

Pour Menuiseries Vasseur, cela produit `CLIENT`, `CHANTIER`, `PRODUIT`, `POSEUR` — et `POSER (#NumChantier, #RefProduit, Quantité)`, née de l'association n:n qui bloquait la réunion.

On reconnaît un bon découpage à trois contrôles, dans cet ordre :

- **1FN** — une seule valeur par case. Pas de colonne « Modèles posés » contenant « F-210, F-315 » ;
- **2FN** — tout attribut non-clé dépend de la **totalité** de la clé. Dans `POSER`, la quantité dépend bien du couple ; le nom du client, lui, n'y aurait rien à faire ;
- **3FN** — pas de dépendance transitive. Si `CHANTIER` portait le code postal *et* la ville, la ville dépendrait du code postal et non du chantier : il faudrait la sortir.

Et le test de bon sens, qui vaut tous les formalismes : **chaque fait doit être écrit à un seul endroit.** Si vous devez corriger la même information à deux endroits pour qu'elle reste

vraie, le modèle est faux — c'est exactement le tableur du début du chapitre, et la facture partie à la mauvaise adresse.

✓ **Vérifiez que vous avez compris**

Un étudiant propose la relation `CHANTIER (NumChantier, DateDebut, #NumClient, NomClient, VilleClient)`. Que lui répondez-vous ?

Qu'elle viole la **troisième forme normale**. `NomClient` et `VilleClient` ne dépendent pas de `NumChantier` : ils dépendent de `#NumClient`, qui n'est pas la clé primaire de cette relation. C'est une **dépendance transitive**.

La conséquence est concrète : l'identité du client serait recopiée sur chacun de ses chantiers, et il faudrait la corriger partout à chaque changement. La correction consiste à ne garder que `#NumClient` dans `CHANTIER`, le nom et la ville restant dans `CLIENT`.

Retenez le raisonnement, pas la formule : dès qu'un attribut non-clé en détermine un autre, il manque une table.

Mise en pratique

Cas pratique 3 : Le modèle de Thomas

Compétences attendues

- Interpréter un schéma relationnel
- Vérifier les règles du modèle relationnel
- Adapter un schéma relationnel à un besoin d'évolution d'une base de données

Thomas Lemoine a pris de l'avance : il propose le modèle suivant pour la future base.

```
CLIENT     (NumClient, NomClient, Categorie, VilleClient)
CHANTIER   (NumChantier, DateDebut, VilleChantier, #NumClient, NomClient,
            Poseurs, MontantHT, MontantTTC)
LIGNE_POSE (NumChantier, RefProduit, Designation, Matiere, Quantite)
```

Dans `CHANTIER`, l'attribut `Poseurs` contient les noms des poseurs intervenus, séparés par des virgules (« Mercier, Aubry »). La clé primaire de `LIGNE_POSE` est le couple (`NumChantier`, `RefProduit`).

Missions

1. **Relevez les manquements aux trois formes normales.** Pour chacun, nommez la forme normale en cause, expliquez la dépendance fautive et décrivez l'anomalie concrète qu'elle provoque.
2. **Proposez un modèle relationnel corrigé**, en respectant la notation vue en cours (clé primaire en tête, clé étrangère précédée d'un `#`).
3. **Dans votre modèle, quelle est la clé primaire de la table qui enregistre les produits posés ? Pourquoi la quantité s'y trouve-t-elle, et nulle part ailleurs ?**
4. **Camille Vasseur veut suivre la facturation : un chantier donne lieu à une ou plusieurs factures (un acompte à la commande, le solde à la fin du chantier), chacune avec un numéro, une date, un type et un montant. Adaptez votre modèle.**

Synthèse

1. **Recensement des données élémentaires** :

- Celui-ci peut se faire dans un dictionnaire de données qui précisera le type, la règle de calcul éventuelle ou toute autre observation utile ;
- Les ambiguïtés sont résolues par l'attribution d'un nom unique pour chaque donnée ;
- Les données élémentaires retenues sont des attributs (plus petit élément d'information manipulable).

2. **Mise en évidence des relations entre les données** :

- Mise en évidence des dépendances fonctionnelles à travers la construction d'un graphe des dépendances fonctionnelles ;
- Élaboration du modèle relationnel à partir du graphe ;
- Construction d'une relation universelle (RU), qui regroupe les données du dictionnaire et donc tous les attributs de la base ;
- Normalisation des relations en trois étapes pour arriver à la représentation du modèle relationnel en troisième forme normale.

3. **Synthèse de toutes les relations dans un modèle relationnel** :

```
RELATION1 (ClePrimaire1, Attribut1A, Attribut1B, …, #CleEtrangere1)
RELATION2 (ClePrimaire2, Attribut2A, Attribut2B, …)
```

- `#CleEtrangere1` est la reprise de `ClePrimaire2` dans `RELATION1` : c'est elle qui assure le lien entre les deux relations ;
- La représentation du modèle peut également se faire sous forme de tables.

Les cinq réflexes à emporter

1. **Un fait, un endroit.** Toute information écrite deux fois finira par se contredire.
2. **On ne stocke pas ce qu'on sait recalculer.** Les totaux, la TVA, l'âge : ce sont des résultats, pas des données.
3. **Une dépendance fonctionnelle est vraie pour toujours**, pas seulement dans l'exemple qu'on a sous les yeux.
4. **Toute association n:n crée une table**, dont la clé primaire est composée des deux clés étrangères.
5. **Un attribut non-clé qui en détermine un autre** signale une table manquante — c'est la 3FN.

◀ Partie 2 : Gérer les données du système d'information · 2.2 Le modèle relationnel avancé ▶